

Hoja de Ejercicios Complementarios: Bloque 3

Ecosistema XML: Modelos DOM, Transformaciones XSLT, Parsers SAX/StAX y Mapeo JAXB

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	DAM 2º Curso · UT1	Presencial / Distancia · Java 21	Alumnado · Consolidación Técnica

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

RA1: Manejo de Ficheros

CE1.e (Formatos de Intercambio de Información: XML)

Este bloque cubre las cuatro grandes estrategias de procesamiento del estándar XML en la industria del software: creación y modificación de árboles en memoria RAM mediante **DOM** y serialización formateada con **Transformer**; procesamiento reactivo en *streaming* mediante eventos con **SAX**; control de cursor bajo demanda mediante **StAX** con salida anticipada; y mapeo bidireccional objeto-relacional declarativo mediante **JAXB**.

Cuadro Resumen: Cuándo Utilizar Cada Tecnología XML

Tecnología	Modelo	Consumo de RAM	Ventajas Principales	Caso de Uso Recomendado
DOM	Árbol en RAM	Alto ($5 \times$ - $10 \times$ el XML)	Acceso bidireccional, inserción/borrado de nodos	Ficheros < 50 MB con modificaciones complejas
SAX	Streaming Push (Eventos)	Mínimo ($O(1)$)	Máxima velocidad, no satura memoria	Ficheros gigantes (> 500 MB) de solo lectura
StAX	Streaming Pull (Cursor)	Mínimo ($O(1)$)	El código manda, permite <code>break</code> anticipado	Búsquedas puntuales y pipelines de transformación
JAXB	Data Binding (Anotaciones)	Medio (Objetos Java)	Sin lidiar con nodos ni parsers manuales	Servicios web, APIs REST/SOAP y configuración

Ejercicio 1: Generador de Facturas Electrónicas DOM con Transformer

(FacturaDOM.java)

NIVEL
BÁSICO

Contexto Empresarial:

La legislación mercantil exige la emisión de facturas electrónicas en formato XML válido y formateado para su validación ante sistemas fiscales. Debes implementar un módulo en Java capaz de instanciar un árbol DOM desde cero y volcarlo a disco con indentación de 4 espacios.

Estructura XML Esperada (datos/factura_electronica.xml):

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<factura numero="FAC-2026-0042" fecha="2026-09-17">
  <emisor cif="B-33123456">
    <nombre>ElectroTech S.L.</nombre>
    <ciudad>Avilés</ciudad>
  </emisor>
  <cliente nif="71888999X">
    <nombre>Desarrollos Astur S.A.</nombre>
    <ciudad>Gijón</ciudad>
  </cliente>
  <lineas>
    <linea id="1">
      <concepto>Servidor Rack 1U</concepto>
      <cantidad>2</cantidad>
      <precioUnitario>850.00</precioUnitario>
    </linea>
    <linea id="2">
      <concepto>Switch Gigabit 24 Puertos</concepto>
      <cantidad>1</cantidad>
      <precioUnitario>145.50</precioUnitario>
    </linea>
  </lineas>
</factura>
```

Instrucciones Técnicas Clave:

1. Crea el documento vacío con `DocumentBuilderFactory.newInstance().newDocumentBuilder().newDocument()`.
2. Construye la jerarquía con `doc.createElement()`, `doc.appendChild()`, `doc.createTextNode()` y `elemento.setAttribute()`.
3. Configura el serializador `Transformer` :
 - `tf.setOutputProperty(OutputKeys.INDENT, "yes");`
 - `tf.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");`
 - `tf.setOutputProperty(OutputKeys.ENCODING, "UTF-8");`
4. Serializa el árbol mediante `tf.transform(new DOMSource(doc), new StreamResult(new File(...)))`.

Salida Esperada en Consola:

[DOM] Construyendo árbol XML en memoria...

[DOM] Raíz '<factura>' y nodos de emisor, cliente y líneas ensamblados.

[TRANSFORMER] Serializando documento a 'datos/factura_electronica.xml' con sangría de 4 espacios.

[ÉXITO] Archivo generado correctamente (1.024 bytes). Inspección visual conforme a normativa.



Ejercicio 2: Modificador de Tarifas DOM In-Situ sin Corrupción por Nodos #text

(`GestorTarifasDOM.java`)

NIVEL

INTERMEDIO

Contexto Empresarial:

El departamento comercial dispone de un catálogo `datos/catalogo_tienda.xml`. Durante las rebajas, se requiere una utilidad que aplique un descuento porcentual (ej. 15%) a todos los artículos pertenecientes a una categoría concreta (ej. "Perifericos"), modificando el XML original directamente en el disco.

Catálogo de Prueba (`datos/catalogo_tienda.xml`):

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
  <producto id="P01" categoria="Hardware">
    <nombre>Disco SSD NVMe 1TB</nombre>
    <precio>89.95</precio>
    <stock>25</stock>
  </producto>
  <producto id="P02" categoria="Perifericos">
    <nombre>Raton Ergonomico Inalambrico</nombre>
    <precio>34.50</precio>
    <stock>12</stock>
  </producto>
  <producto id="P03" categoria="Perifericos">
    <nombre>Teclado Mecanico RGB</nombre>
    <precio>65.00</precio>
    <stock>8</stock>
  </producto>
</catalogo>
```

Requerimientos Técnicos:

1. Carga el XML con `DocumentBuilder.parse(archivo)` y normaliza con `doc.getDocumentElement().normalize()`.
2. Obtén la lista de nodos con `doc.getElementsByTagName("producto")`.
3. **Filtrado de Nodos:** Al iterar, comprueba siempre `if (nodo.getNodeType() == Node.ELEMENT_NODE)` antes de castear a `Element`.
4. Si `producto.getAttribute("categoria").equalsIgnoreCase(categoriaObjetivo)`:
 - Localiza la etiqueta `<precio>` mediante `producto.getElementsByTagName("precio").item(0)`.
 - Calcula el precio rebajado y actualiza el texto del nodo con `nodoPrecio.setTextContent(String.format(Locale.US, "%.2f", nuevoPrecio))`.
5. Sobrescribe el archivo en disco mediante `Transformer` preservando la indentación de 4 espacios.

La Trampa de los Nodos de Texto Whitespace (#text)

En el estándar W3C DOM, los saltos de línea e indentaciones entre etiquetas se interpretan como nodos hijos de tipo texto (`#text`). Si intentas castear directamente un `Node` a `Element` sin comprobar `getNodeTypes() == Node.ELEMENT_NODE`, la JVM lanzará un `ClassCastException` instantáneo.

Salida Esperada en Consola:

```
[INFO] Aplicando 15,00% de descuento a productos de categoría 'Perifericos'...  
-> Modificado 'Raton Ergonomico Inalambrico': 34.50 € -> 29.32 €  
-> Modificado 'Teclado Mecanico RGB': 65.00 € -> 55.25 €  
[INFO] 'Disco SSD NVMe 1TB' omitido (Categoría: Hardware).  
[ÉXITO] Archivo 'datos/catalogo_tienda.xml' actualizado in-situ. 2 productos rebajados.
```

Ejercicio 3: Analizador Dual Streaming: Reactivo SAX vs Cursor StAX

(AnalizadorDualXML.java)

NIVEL
AVANZADO

Contexto Empresarial:

En pasarelas de pago o en la Agencia Tributaria se procesan ficheros XML de **500 MB a 2 GB**. Parsear estos volúmenes con DOM causaría un desbordamiento inmediato de memoria (`OutOfMemoryError`). Se requiere desarrollar dos soluciones basadas en flujos continuos (*streaming*):

Parte A: Parser Reactivo SAX (`CatalogoStatsHandler.java`):

- Crea una clase que herede de `org.xml.sax.helpers.DefaultHandler`.
- Calcula en una única pasada secuencial hacia adelante:
 1. Número total de productos en el catálogo.
 2. Valor total de inventario ($\sum \text{precio} \times \text{stock}$).
 3. Nombre y precio del artículo más caro.
- **Regla de oro de SAX:** El método `characters(char[] ch, int start, int length)` puede fragmentar el texto en múltiples llamadas. Acumula siempre los caracteres en un `StringBuilder` y haz la conversión numérica únicamente en el evento `endElement()`.

Parte B: Cursor Pull StAX con Salida Anticipada (`BuscadorStAX.java`):

- Instancia `XMLInputFactory.newInstance().createXMLStreamReader(new FileInputStream(archivo))`.
- Implementa el método `buscarProductoPorId(File archivo, String idBuscado)`.
- Avanza el cursor mediante `while (reader.hasNext())` y `reader.next()`.
- En cuanto detectes el evento `XMLStreamConstants.START_ELEMENT` correspondiente a un `<producto>` con el ID buscado, extrae sus datos y **ejecuta un `break` para detener el bucle de inmediato**.
- Demuestra que StAX no necesita leer los 500 MB restantes si el elemento deseado se encontraba en la cabecera del documento.

Salida Esperada en Consola:

```
=== PARTE A: ANÁLISIS AGREGADO SAX (1 Pasada Streaming) ===
Total productos procesados: 3
Valor total del inventario: 3.181,50 €
Producto con mayor precio: Disco SSD NVMe 1TB (89,95 €)
Memoria RAM consumida por el parser: < 2 MB (O(1))

=== PARTE B: BÚSQUEDA SELECTIVA StAX (Pull Cursor) ===
Buscando producto con ID 'P02'...
-> ¡Encontrado en el evento 18! Nombre: Raton Ergonomico Inalambrico | Stock: 12
[STAX BREAK] Bucle detenido inmediatamente. No se procesó el resto del archivo.
```

Contexto Empresarial:

Una distribuidora de libros digitales necesita intercambiar su catálogo entre microservicios Java. En lugar de manipular nodos o cursores manualmente, se exige utilizar la API declarativa estándar **Jakarta XML Binding (JAXB)** mediante POJOs y anotaciones.

Requerimientos Técnicos y Modelado:

1. Clase **Libro** :

- Anotaciones: `@XmlRootElement(name = "libro")` y `@XmlType(propOrder = {"titulo", "autor", "precio", "paginas"})`.
- Atributo XML: `@XmlAttribute(name = "isbn") private String isbn;`.
- Elementos hijos: `titulo` (String), `autor` (String), `precio` (double), `paginas` (int).
- **Constructor obligatorio:** Debe incluir constructor público sin parámetros (`public Libro() {}`) para que el motor de introspección de JAXB pueda instanciarlo.

2. Clase **Biblioteca** :

- Anotación raíz: `@XmlRootElement(name = "biblioteca")`.
- Atributo XML: `@XmlAttribute(name = "nombre") private String nombre;`.
- Lista de libros: `@XmlElement(name = "libro") private List<Libro> libros;`.
- Constructor sin parámetros y métodos getter/setter.

3. **Exportación (Marshalling):** Instancia `JAXBContext context = JAXBContext.newInstance(Biblioteca.class);`, crea el `Marshaller`, activa `marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);` y guarda en `datos/biblioteca.xml`.

4. **Importación (Unmarshalling):** Utiliza `Unmarshaller unmarshaller = context.createUnmarshaller();` para reconstruir el objeto `Biblioteca` desde el archivo XML y mostrar en consola el listado completo de libros con sus precios.

Requisito Estricto de JAXB: Constructores por Defecto

Si una clase anotada con JAXB carece de un constructor público sin argumentos (`public MiClase() {}`), el `JAXBContext` fallará en tiempo de ejecución con una excepción fatal: `NoSuchMethodException: ...<init>()`.

Salida Esperada en Consola:

```
[JAXB MARSHALLING] Serializando instancia de Biblioteca 'Biblioteca Central Avilés'...
-> Archivo 'datos/biblioteca.xml' generado con sangría y codificación UTF-8.
```

```
[JAXB UNMARSHALLING] Reconstruyendo objetos Java desde XML...
```

```
Biblioteca: Biblioteca Central Avilés
```

- [978-84-12345] "El Quijote" de Miguel de Cervantes | 24,95 € (860 págs.)
- [978-84-67890] "Clean Code" de Robert C. Martin | 42,50 € (464 págs.)
- [978-84-55443] "Effective Java" de Joshua Bloch | 49,00 € (412 págs.)

```
[ÉXITO] Mapeo bidireccional POJO-XML completado sin manipulaciones manuales de nodos.
```

